浙江大学

ZJUI 学院 2025 年暑期科研项目总结材料

中文论文题目 ：基于平均流模型的一步去噪

英文论文题目：One-step Denoising with MeanFlow Model

成员姓名：郑涵缤（3240110753）

徐嘉铭（3240110789）

丁昊（3240110961）

指导教师：孟祥明

指导教师所在学院：ZJUI 学院

暑期研究起止日期：2025 年 06 月 15 日-2025 年 08 月 07 日

摘要

本工作探索了平均流一步生成式模型（MeanFlow）在图像去噪任务中的应用。本研究的主要目标是理解流匹配生成式模型（Flow Matching）及其优化方案 MeanFlow 模型的基本原理，并在 MNIST 与 CIFAR-10 数据集上进行实验。在此基础上，我们将 MeanFlow 模型改造用于图像去噪任务：不再以简单的高斯分布作为初始分布，而是以加噪声的真实图像作为起点，通过 MeanFlow 模型的一步生成过程对图像进行去噪。MNIST 实验表明，一次函数评估（NFE = 1）即可显著降低噪声，使峰值信噪比（PSNR）从约 6 dB 提升至 22.6 dB。二次函数评估（NFE=2）将 PSNR 进一步提升至约 24 dB，而更多推理步骤增益有限。实验结果表明，MeanFlow 模型能以极低的推理成本有效完成图像去噪任务。本工作未提出新的算法或模型框架，而是对一步生成模型在图像去噪任务中的应用进行了实践探索，初步验证了 MeanFlow 方法在快速图像去噪中的可行性。

Abstract

This work investigates the application of the MeanFlow one-step generative model to image denoising tasks. The primary objective of this summer research is to understand the fundamental principles of Flow Matching generative modeling and its improved version, the MeanFlow model, through theoretical derivation and experiments. Based on this understanding, we specifically adapt the MeanFlow model for image denoising by modifying its initial distribution: from simple distributions like Gaussian to noisy images, retaining the model's one-step generation capacity while serving the denoising purpose. Experiments on MNIST dataset show that a single function evaluation (NFE = 1) significantly reduces the noise, increasing the Peak Signal-to-Noise Ratio (PSNR) from approximately 6 dB to 22.6 dB. A second evaluation (NFE = 2) further improves the PSNR to 24 dB, while additional sampling steps yield limited gains. This work demonstrates that the MeanFlow framework is capable of performing effective image denoising at minimal sampling cost, positioning it as a practical alternative to conventional multi-step denoising methods. Although our work does not propose new algorithms or model frameworks, it explores the application of the MeanFlow framework to image denoising, verifying its feasibility for future image restoration tasks.

1 Introduction

Generative modeling aims to transform a simple prior distribution into a complex target data distribution. Among existing approaches, flow-based models provide an understandable method by explicitly defining an invertible transformation between random variables and computing the corresponding Jacobian determinants. In contrast, the Flow Matching framework [3] achieves this by modeling an instantaneous velocity field that transports one distribution into another based on the continuity equation. Both approaches are built on concepts of "flows" and construct an explicit mapping from the prior to the target distribution. However, they differ in what the transform is and how it is learned.

Recently, research has paid more attention to few-step, and particularly, one-step generative models which offer significant sampling advantages. The MeanFlow model [1] extends Flow Matching by introducing a ground-truth average velocity field derived from the instantaneous velocity field in Flow Matching. The average velocity is defined as the ratio of displacement to a time interval, with the displacement computed as the integral of the instantaneous velocity. This formulation allows the neural network to directly learn from the well-defined average velocity without any extra consistency heuristic [1], enabling reliable and effective one-step generation. Moreover, classifier-free guidance (CFG) can be incorporated into the target vector field without additional cost at sampling time, preserving model's efficiency.

In this work, we focus on Flow Matching and MeanFlow. Our primary goal is to understand their fundamental principles through theoretical derivation and hands-on experiments. Building on this foundation, we adapt the MeanFlow framework for one-step image denoising task. Specifically, we alters its initial distribution to noisy images instead of gaussian noise, retaining its one-step generation ability and serving the denoising purpose. Experiments on MNIST demonstrate that MeanFlow denoising method can substantially reduce noise with just a single function evaluation, highlighting its potential as an efficient one-step denoising method.

2 Background

2.1 Flow Based Model

Flow-based models define an invertible transformation between a simple prior distribution and a target data distribution.

Let x denote a random variable following a simple prior distribution, such as the standard Gaussian, and let y denote a variable following the target distribution. Then there exists a mapping $f : \mathbb{R}^d \to \mathbb{R}^d$ such that $y = f(x)$. By the change of variable formula, the probability density funciton of x and y relates with:

$$p_Y(y) = p_X(x)\frac{\partial x}{\partial y}, \tag{1}$$

where $\dfrac{\partial x}{\partial y}$ is the Jacobian matrix of the inverse transformation f^{-1}.

With this formula, we can learn the mapping function f directly, and this explicit relationship makes the flow-based model interpretable. However, computing the Jacobian determinant is usually computational expensive, and this constrains the model design.

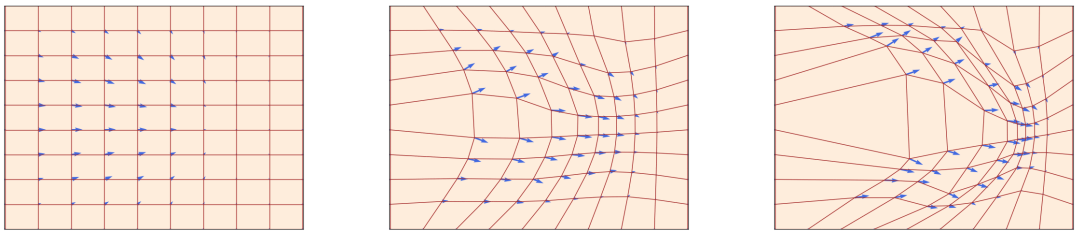

Figure 1: Illustration of flows. Here, the evolution of the density is defined by a velocity field $v : \mathbb{R}^d \to \mathbb{R}^d$ (blue arrows) specifying its instantaneous movements at all locations (here $d = 2$). Figure from [4].

2.2 Flow Matching

2.2.1 Continuity Equation and Velocity Field

The continuity equation describes how a density evloves with a velocity field. Let $v(x,t)$ and $\rho(x,t)$ denote the velocity field and the density of a substance at position x and time t. The continuity equation is given by:

$$\frac{\partial \rho(x,t)}{\partial t} + \nabla_x \cdot (\rho(x,t)v(x,t)) = 0. \tag{2}$$

It states that, for any infinitesimal volume, the change of the density inside the volume equals the net flux across the boundary. This local balance, under some reasonable assumptions like the convergence of the global density, implies the global conservation of the substance. In our case, the "total substance" corresponds to the probability mass, and $\rho(x,t)$ represents the probability density. As a distribution evloves from a prior to the target data distribution, the continuity equation motivates the use of a velocity field to describe this evolution.

Let p_{prior} and p_{target} denote the prior and the target distribution, and let $x_0 \sim p_{\text{prior}}$ and $x_1 \sim p_{\text{target}}$. Suppose the distribution evolves from $t = 0$ to $t = 1$ with intermediate distribution p_t and $x_t \sim p_t$, we defined the velocity field $v(x_t, t)$ as:

$$v(x_t,t) \triangleq \frac{dx_t}{dt}. \tag{3}$$

Given $v(x_t, t)$, a sample from p_t can be obtained by:

$$x_t - x_0 = \int_0^t v(x_\tau, \tau)\, d\tau. \tag{4}$$

In particular, selecting $t = 1$ produces a sample from p_{target}.

2.2.2 Probability Path and the Training Target

A deterministic trajectory from p_{prior} to p_{target} is called a probability path or marginal probability path. This evolution is driven by the marginal velocity field $v(x_t, t)$. However, directly obtaining $v(x_t, t)$ is generally difficult. Flow Matching constructs an equivalent velocity $v^{\text{tgt}}(x_t, t)$ with the weighted average.

Consider sampling z from p_{target} and defining a conditional probability path from p_{prior} to δ_z with conditional distribution $p_t(\cdot \mid z)$:

$$x_t = (1-t)x_0 + tz. \tag{5}$$

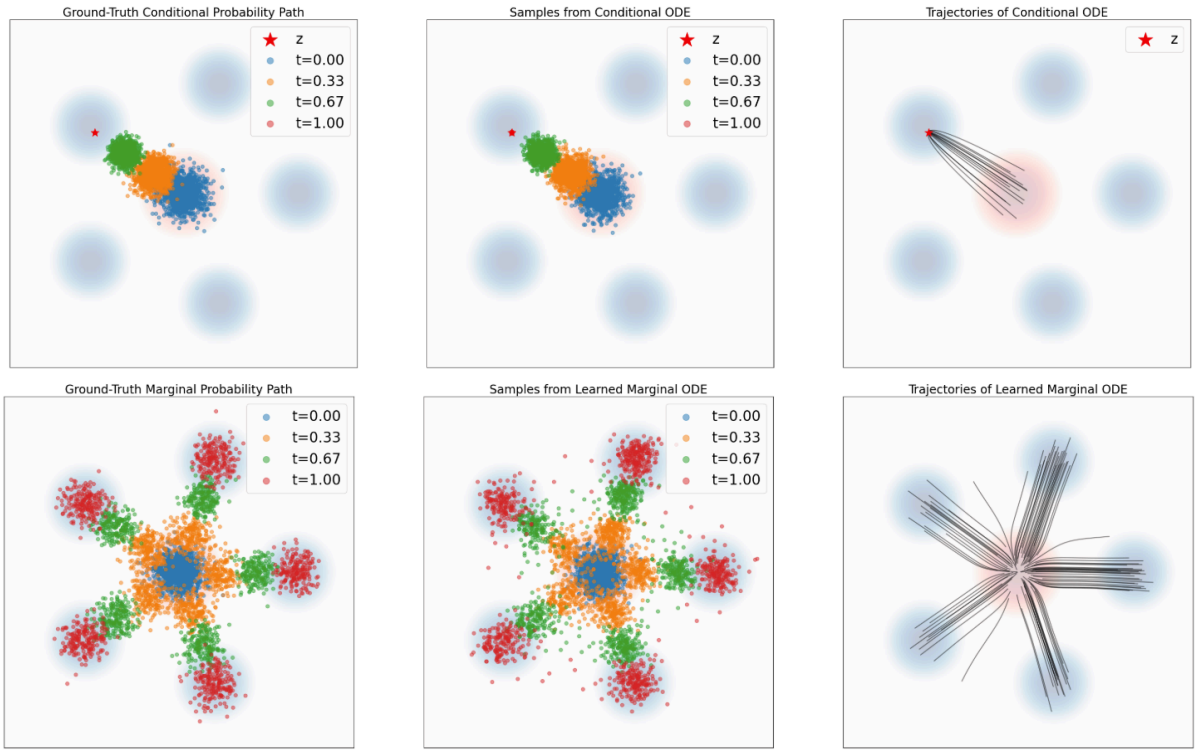

Figure 2: Illustration of conditional and marginal probability path. Target distribution p_{target} in blue background. Gaussian prior distribution p_{prior} in red background. Top row: Conditional probability path. Left: Ground truth samples from $p_t(\cdot \mid z)$. Middile: ODE samples over times. Right: Trajectories by simulating ODE in Equation (6). Bottom row: Simulating a marginal probability path. Left: Ground truth samples from p_t. Middle: ODE samples over times. Right: Trajectories by simulating ODE in Equation (3). As one can see, the conditional vector field follows the conditional probability path and the marginal vector field follows the marginal probability path. Figure from [2].

The marginal distribution is then given by the total probability throrem:

$$p_t(x_t) = \int p_t(x_t \mid z) p_{\text{target}}(z)\, dz.$$

The corresponding conditional velocity field is:

$$v(x_t, t \mid z) \triangleq \frac{dx_t}{dt} = z - x_0. \tag{6}$$

The marginal velocity, which serves as the training target, is constructed by taking the expectation of the conditional velocity under the weights of z:

$$\begin{aligned} v^{\text{tgt}}(x_t, t) &\triangleq \int v(x_t, t \mid z) p_t(z \mid x_t)\, dz \\ &= \int v(x_t, t \mid z) \frac{p_t(x_t \mid z) p_{\text{target}}(z)}{p_t(x_t)}\, dz. \end{aligned} \tag{7}$$

The validity of this construction with respect to $v^{\text{tgt}}(x_t, t)$ is discussed in Appendix A.

2.2.3 Train the Model

Our generative model is trained by fitting a neural network $v^\theta(x_t, t)$ to approximate the target velocity field $v^{\text{tgt}}(x_t, t)$. A natural choice is the mean squared error (MSE) loss:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t, x_t \sim p_t} \left[\left\| v^\theta(x_t, t) - v^{\text{tgt}}(x_t, t) \right\|_2^2 \right] \tag{8}$$

However, there is no closed-form for $v^{\text{tgt}}(x_t, t)$, and what we have access to is only samples from p_{target} and p_{prior}. We now show that, under the MSE loss, one may equivalently replace $v^{\text{tgt}}(x_t, t)$ with the conditional velocity field $v(x_t, t \mid z)$:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, z \sim p_{\text{target}}, x_t \sim p_t(\cdot|z)} \left[\left\| v^\theta(x_t, t) - v^{\text{tgt}}(x_t, t \mid z) \right\|_2^2 \right], \tag{9}$$

and the two objectives share the same gradients:

$$\nabla_\theta \mathcal{L}_{\text{CFM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{FM}}(\theta). \tag{10}$$

Proof. Expanding the square in $\mathcal{L}_{\text{FM}}$ and $\mathcal{L}_{\text{CFM}}$, we obtain

$$\begin{aligned}
\mathcal{L}_{\text{FM}}(\theta) &= \mathbb{E}_{t, x_t \sim p_t} \left[\left\| v^\theta(x_t, t) \right\|_2^2 \right] - 2\mathbb{E}_{t, x_t \sim p_t} \left[\left\| v^\theta(x_t, t)^\top v^{\text{tgt}}(x_t, t) \right\|_2^2 \right] + \mathbb{E}_{t, x_t \sim p_t} \left[\left\| v^{\text{tgt}}(x_t, t) \right\|_2^2 \right] \\
\mathcal{L}_{\text{CFM}}(\theta) &= \mathbb{E}_{t, z \sim p_{\text{target}}, x_t \sim p_t(\cdot|z)} \left[\left\| v^\theta(x_t, t) \right\|_2^2 \right] - 2\mathbb{E}_{t, z \sim p_{\text{target}}, x_t \sim p_t(\cdot|z)} \left[v^\theta(x_t, t)^\top v^{\text{tgt}}(x_t, t \mid z) \right] \\
&\quad + \mathbb{E}_{t, z \sim p_{\text{target}}, x_t \sim p_t(\cdot|z)} \left[\left\| v^{\text{tgt}}(x_t, t \mid z) \right\|_2^2 \right]
\end{aligned}$$

For the cross term, we compute

$$\begin{aligned}
\mathbb{E}_{t, x_t \sim p_t} \left[v^\theta(x_t, t)^\top v^{\text{tgt}}(x_t, t) \right] &= \iint v^\theta(x_t, t)^\top v^{\text{tgt}}(x_t, t) p_t(x_t)\, dt\, dx_t \\
&= \iint v^\theta(x_t, t)^\top \left(\int v(x_t, t \mid z) \frac{p_t(x_t \mid z) p_{\text{target}}(z)}{p_t(x_t)}\, dz \right) p_t(x_t)\, dt\, dx_t \\
&= \iiint v^\theta(x_t, t)^\top v(x_t, t \mid z) p_t(x_t \mid z) p_{\text{target}}(z)\, dz\, dt\, dx_t \\
&= \mathbb{E}_{t, z \sim p_{\text{target}}, x_t \sim p_t(\cdot|z)} \left[v^\theta(x_t, t)^\top v(x_t, t \mid z) \right]
\end{aligned}$$

The expectation of $v^\theta(x_t, t)$ are identical under two different measures, thus $\mathcal{L}_{\text{FM}}$ and $\mathcal{L}_{\text{CFM}}$ differ only in terms independent of θ. Since such terms vanish under gradient, $\nabla_\theta \mathcal{L}_{\text{CFM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{FM}}(\theta)$.

More generally, for any error function that is linear in $v^{\text{tgt}}(x_t, t)$, one may substitute $v^{\text{tgt}}(x_t, t)$ with $v(x_t, t \mid z)$ without affecting the optimization (see Appendix B).

3 Methodology

3.1 Mean Flow

Following [1], the instantaneous velocity field in Flow Matching can be replaced with an average velocity to enable one-step generation. Specifically, from time r to time t, the average velocity is defined as:

$$u(x_r, r, t) \triangleq \frac{1}{t-r}\int_r^t v(x_\tau, \tau)\, d\tau. \tag{11}$$

Rewrite Equation (11) as $(t-r)u(x_r, r, t) = \int_r^t v(x_\tau, \tau)\, d\tau$, and differentiate both sides with respect to r:

$$u(x_r, r, t) = v(x_r, t) + (t-r)\frac{d}{dr}u(x_r, r, t),$$

Applying the total derivative theorem, we obtain:

$$\begin{aligned} u(x_r, r, t) &= v(x_r, r) + (t-r)\left(\frac{\partial u}{\partial x_r}\frac{dx_r}{dr} + \frac{\partial u}{\partial t}\frac{dt}{dr} + \frac{\partial u}{\partial r}\frac{dr}{dr}\right) \\ &= v(x_r, r) + (t-r)\left(\frac{\partial u}{\partial x_r}v(x_r, r) + \frac{\partial u}{\partial r}\right) \end{aligned} \tag{12}$$

since $\frac{dt}{dr} = 0$ and $\frac{dx_r}{dr} = v(x_r, r)$.

Similiar to Flow Matching, we train a neural network $u^\theta(x_r, r, t)$ to approximate $u(x_r, r, t)$ with an MSE loss:

$$\mathcal{L}_{\text{MF}}(\theta) = \mathbb{E}_{r,t,x_r\sim p_r}\left[\left\|u^\theta(x_r, r, t) - \text{sg}\left[u^{\text{tgt}}(x_r, r, t)\right]\right\|_2^2\right], \tag{13}$$

where $\text{sg}[\cdot]$ denotes the stop gradient operator and $u^{\text{tgt}}(x_r, r, t)$ is given by:

$$u^{\text{tgt}}(x_r, r, t) = v^{\text{tgt}}(x_r, r) + (t-r)\left(\frac{\partial u^\theta}{\partial x_r}v^{\text{tgt}}(x_r, r) + \frac{\partial u^\theta}{\partial r}\right). \tag{14}$$

Here, the derivatives of u are approximated by those of u^θ, following [1].

Since the error term $u^\theta(x_r, r, t) - \text{sg}\left[u^{\text{tgt}}(x_r, r, t)\right]$ is a linear function of $v^{\text{tgt}}(x_r, r)$, and according to the result in Appendix B, this allows $v^{\text{tgt}}(x_r, r)$ to be substituted with the conditional velocity $v(x_r, r \mid z)$:

$$\mathcal{L}_{\text{CMF}}(\theta) = \mathbb{E}_{r,t,z\sim p_{\text{target}},x_r\sim p_r(\cdot|z)}\left[\left\|u^\theta(x_r, r, t) - \text{sg}\left[u^{\text{cond}}(x_r, r, t)\right]\right\|_2^2\right] \tag{15}$$

$$u^{\text{cond}} = v(x_r, r \mid z) + (t-r)\left(\frac{\partial u^\theta}{\partial x_r}v(x_r, r \mid z) + \frac{\partial u^\theta}{\partial r}\right) \tag{16}$$

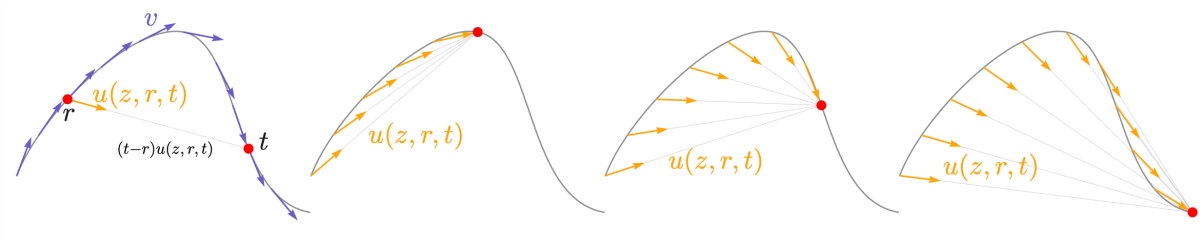

Figure 3: Illustration of the average velocity $u(z, r, t)$. Here, z means the current location, i.e. x_r in Equation (11). Leftmost: The instantaneous velocity $v(x_t, t)$. Right three subplots: The average velocity field at different time t. Figure revised from [1].

This formulation allows MeanFlow to be trained similar to Flow Matching, while directly targeting on the average velocity field to enable the one-step generation. The derivative term $\frac{du^\theta}{dr}$ is computed using the Jacobian-Vector-Product (JVP) operation, such as `torch.func.jvp` in PyTorch. Besides, classifier-free guidance (CFG) is also incoporated into the MeanFlow. According to [1], the CFG conditional MSE loss is defined as:

$$\mathcal{L}_{\text{CMF}}^{\text{cfg}}(\theta) = \mathbb{E}_{r,t,x_r\sim p_r}\left[\left\|u_{\text{cfg}}^{\theta}(x_r, r, t \mid c) - \text{sg}\left[u_{\text{cfg}}^{\text{cond}}(x_r, r, t \mid c)\right]\right\|_2^2\right] \tag{17}$$

$$u_{\text{cfg}}^{\text{cond}}(x_r, r, t \mid c) = v_{\text{cfg}}(x_r, r \mid c, z) + (t-r)\left(\frac{\partial u^\theta}{\partial x_r} v_{\text{cfg}}(x_r, r \mid c, z) + \frac{\partial u^\theta}{\partial r}\right), \tag{18}$$

where the CFG-enhanced conditional velocity $v_{\text{cfg}}(x_r, r \mid c, z)$ is given by:

$$v_{\text{cfg}}(x_r, r \mid c, z) = \omega v(x_r, r \mid z) + (1-\omega) u_{\text{cfg}}^{\theta}(x_r, r, r) \tag{19}$$

The details are provided in Appendix C.

3.2 Change the Prior for Denoising

To serve the denoising purpose, we alter the p_{prior} to be noisy images rather than simple distributions like the standard Gaussian. This modification enables the average velocity to directly model the transformation from noisy images to clean images.

3.3 DiT Architecture

We adopt the Diffusion Transformer (DiT) Architecture [5] to parameterize the average velocity field. The input x_t is divided into patches, linearly embedded, and encoded with sinusoid positional embeddings. Time informations (i.e., r and t) are embedded by a lightweight MLP, while class labels are embedded with label embeddings. All embeddings are added to each transformer block through adaptive LayerNorm (adaLN), allowing time and conditional information to modulate both attention and feed-forward layers. The network backbone consists of stacked DiT blocks with multi-head self-attention and GELU-MLPs, followed by a final projection and unpatchify step.

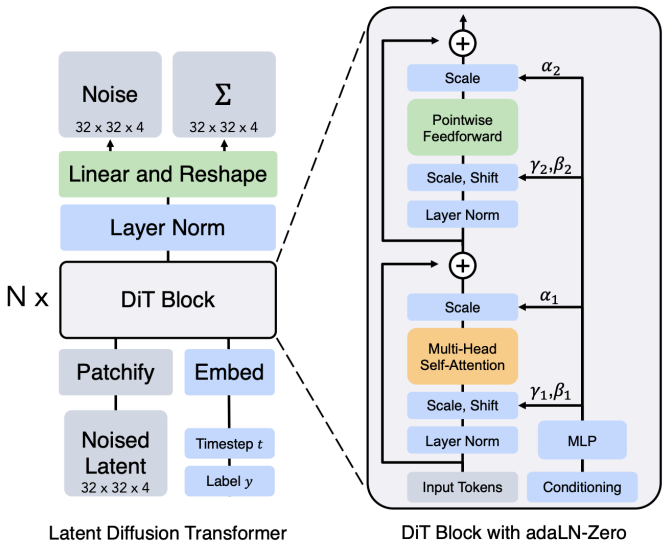

Figure 4: Illustration of the DiT architecture. Figure from [5]

4 Experiment

The main goal of experiments is to test the denoising ability of the MeanFlow framework. We hope to verify that the MeanFlow model can generate clean images from noisy images with only a single function evaluation.

4.1 Experiment Setup

Dataset. We conduct experiments on the MNIST dataset. Noisy inputs (from p_{prior}) are generated by adding Gaussian noise to the original clean images: noisy-img $=$ clean-img $+ \sigma\epsilon$, where ϵ is a standard Gaussian noise and σ is a random variable uniformly distributed in Uniform$(0.3, 0.8)$. We choose this range for σ because when $\sigma > 1$, the noisy input becomes almost unrecognizable and resembles pure Gaussian noise, and when $\sigma < 0.2$, the images remain clear, leaving little need for denoising.

Model Configuration. We adopt the DiT structure described in Section 3.3 to parameterize the average velocity field. In this experiment, the model is set with an input size of $(1 \times 32 \times 32)$, a patch size of (2×2), an embedding dimension $d = 144$, 6 DiT blocks, and 3 attention heads per block. Time and label information are combined via a lightweight MLP and added to each transformer block using adaptive LayerNorm (adaLN).

Loss Metric. We adopt the adaptive L2 loss following [1]. For a regression error Δ, the loss is computed with: $L = \text{sg}[w] \cdot \|\Delta\|_2^2$, where $w = \dfrac{1}{(\|\Delta\|_2^2 + c)^p}$ and $p = 1 - \gamma$. In our experiments, we set $\gamma = 0.5$ and $c = 10^{-3}$. This setting down-weights large errors and stabilizes training.

Sampling (r, t). For each training step, time indices (r, t) are sampled from a logit-normal distribution following [1]. Specifically, we draw two samples from $\mathcal{N}(\mu, \sigma)$ (with $\mu = -0.4$, $\sigma = 1.0$), and pass them through a logistic sigmoid to map them into $(0, 1)$. Then the larger one is assigned to t, and the smaller one is assigned to r. We set a certain portion of random samples with $t = r$.

Training Details. The model is optimized using AdamW with an initial learning rate of 1×10^{-4}, no weight decay, and a batch size of 128, for a total of $300k$ update steps. The learning rate is scheduled by cosine annealing with warm restarts. The initial period is $10k$ steps and doubles after each restart, with a lower bound of 1×10^{-6}. Mixed-precision training (FP16) is employed through `accelerate.Accelerator` to reduce memory consumption and improve efficiency. During training, intermediate checkpoints, denoised samples, and the denoising performance (PSNR) are regularly saved.

Algorithm 1: Training MeanFlow DiT

foreach *Batch of* $(x_0, z) \in$ *Dataset* **do**
- sample (r, t) ;
- compute $x_r = (1 - r)x_0 + rz$;
- compute $v(x_r, r \mid z) = z - x_0$;
- compute $u, du/dr$ with `jvp` ;
- $u^{\text{tgt}} = v - (t - r)\frac{du}{dr}$;
- error $e = u - \text{sg}[u^{\text{tgt}}]$;
- compute loss $\mathcal{L}(\theta)$;
- update θ ;

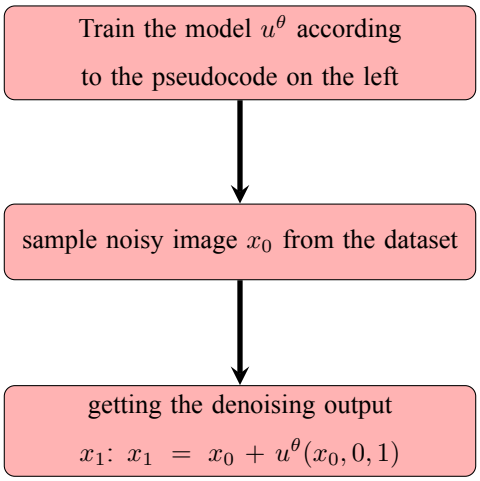

Figure 5: Total Pipeline

4.2 Results and Analysis

Our MeanFlow denoising model is evaluated on the MNIST dataset. Figure 6 shows the PSNR values of 1-NFE denoising result during training, indicating that the model consistently improves as the number of update steps increases. The learning rate and loss during training are provided in Appendix D.

Table 1 summarizes PSNR values at different training stages, confirming steady enhancement of denoising quality over time. Table 2 presents the denoising performance after 300k training steps under different numbers of function evaluations (NFE). Increasing NFE from 1 to 2 yields significant gains, while further increases produce little improvements. This suggests a performance saturation at higher NFEs.

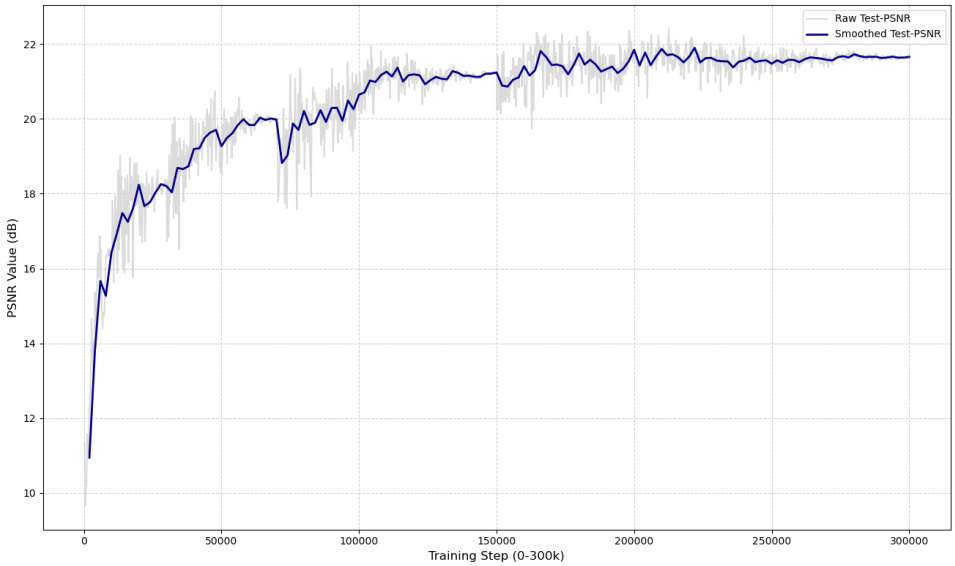

Figure 6: PSNR values over training steps for 1-NFE denoising.

Training Step	PSNR (dB)
Noisy Input	5.9312
2k	12.3043
20k	17.7944
100k	20.2236
200k	21.6694
300k	21.6807

Table 1: PSNR over training steps.

NFE	PSNR (dB)
Noisy Input	6.0401
1	22.6455
2	24.0686
3	24.0816
4	24.0691
5	24.0558

Table 2: Denoising performance under different NFE.

Visual comparison of denoising results under different NFEs are provided in Figure 7. The top row shows the ground truth images, followed by the noisy inputs, and denoised outputs after 1 to 5 NFEs. It's observed that even a single NFE produces notable noise reduction.

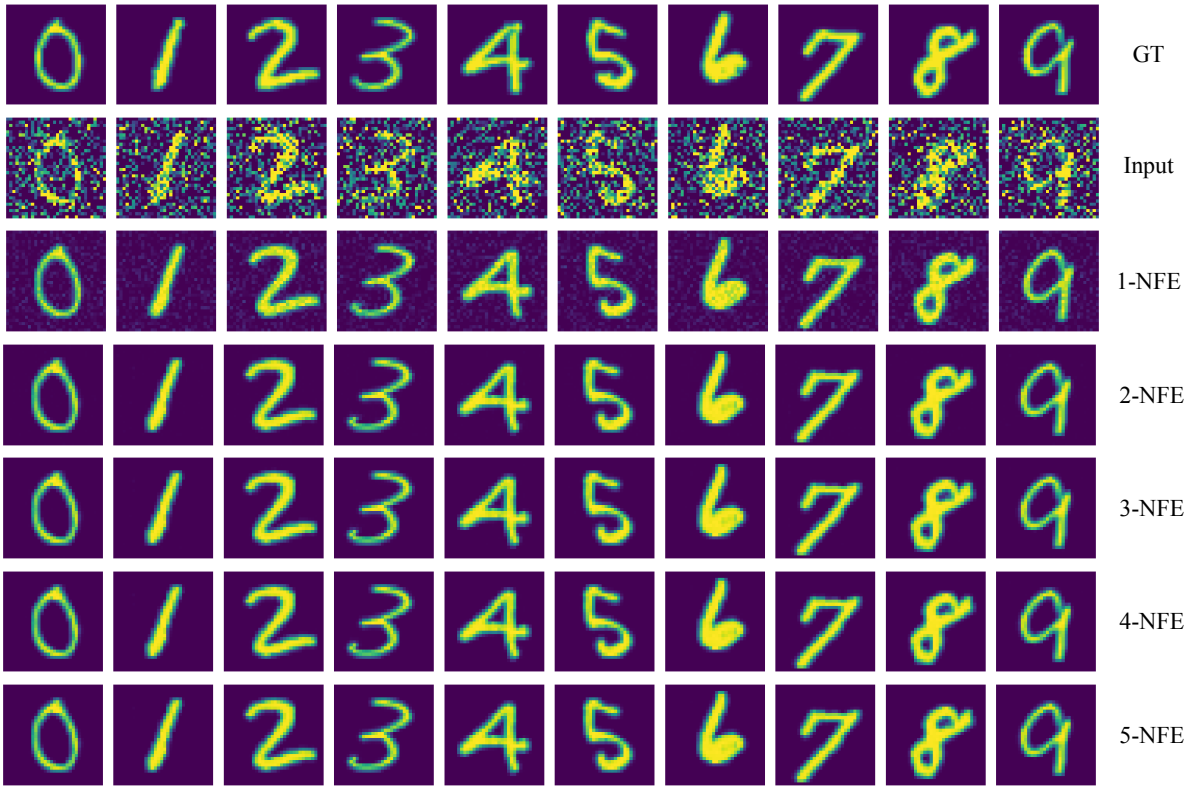

Figure 7: Visual comparison of denoising results under different NFEs. From top to bottom: ground truth, noisy input, and model outputs after 1 to 5 NFE steps.

5 Discussion

5.1 Compatibility with Gaussian Noise

A key factor behind the success of MeanFlow in our experiments lies in the compatibility between Gaussian noise and the flow-based generative modeling framework. Gaussian noise is smooth, unimodal, and isotropic, ensuring that the corrupted data distribution remains mathematically well-behaved (like smooth and differentiable), as required by the continuity equation underlying Flow Matching and MeanFlow. In our experiments, noisy inputs are generated by noisy-img $=$ clean-img $+ \sigma\epsilon$, which corresponds to a pixel-wise affine transformation of independent one-dimensional Gaussians. This preserves the smoothness and structure of the distribution, guaranteeing that the induced velocity fields remain continuous and amenable to stable transport. Conversely, when we clip each pixel of the noisy images to $[0, 1]$ before training, the probability mass of pixels exceeding boundaries accumulates at two ends, creating sharp discontinuities and δ-like densities at the boundaries. Such modification breaks the smoothness assumption, violates the continuity requirement, and finally leads to failure, illustrating the importance of a well-behaved data distribution for both Flow Matching and MeanFlow.

5.2 Architectural Considerations

Our adoption of the Diffusion Transformer (DiT) as the backbone network introduces additional insights. Unlike convolutional networks that emphasize local receptive fields, transformers leverage self-attention to capture global dependencies across the entire image. This ability is particularly beneficial when dealing with Gaussian

noise, which corrupts all pixels uniformly. By allowing information from distant parts of the image to influence local denoising decisions, DiT helps preserve the overall digit structure while recovering fine details. Nevertheless, transformers incur higher computational costs compared to CNNs or UNet, raising questions about scalability. Future work should explore trade-offs between global context modeling and efficiency, potentially through hybrid CNN-transformer architectures.

5.3 Limitations and Future Directions

Despite promising results, the present study has several limitations for further investigation. First, experiments are restricted to MNIST, a relatively simple dataset. Extending to more complex datasets such as CIFAR-10 or Tiny-ImageNet would better demonstrate scalability. Second, only Gaussian noise is considered, whereas real-world noise often exhibits non-Gaussian or structured characteristics (e.g., sensor noise, occlusions). Finally, we have not conducted a systematic computational comparison between MeanFlow and other denoising methods. From the model perspective, the current architecture also presents challenges. While MeanFlow achieves efficient sampling, the JVP operation incurs significant computational overhead during training. Additionally, the fixed guidance scale in CFG somewhat limits its flexibility. Future work would address these limitation by exploring more computationally efficient and effective training target, adaptive guidance strategies, and testing the framework on larger and more diverse datasets with complex noise patterns. Such work will provide a more comprehensive understanding of MeanFlow's abilities and applications.

Conclusion

This work demonstrates the feasibility of adapting the MeanFlow one-step generative modeling for image denoising. By replacing the prior distribution with noisy images, the model effectively transforms corrupted inputs into clean outputs, achieving substantial PSNR gains with minimal function evaluations. Experiments on MNIST show that a single evaluation provides significant noise reduction, while additional steps over 2-NFE offer few returns. This success relies on the smoothness of Gaussian noise and the capacity of the DiT to capture global dependencies and preserve image structure. Limitations include restriction to MNIST, focuses beyond Gaussian noise, computational overhead from JVP and fixed CFG settings. Future work should explore more complex datasets, non-Gaussian noise, efficient training targets, and adaptive guidance strategies. Overall, this study confirms that MeanFlow can serve as an efficient, one-step denoising approach and lays groundwork for broader image restoration applications

References

[1] Zhengyang Geng, Mingyang Deng, Xingjian Bai, J. Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling, 2025.

[2] Peter Holderrieth and Ezra Erives. An introduction to flow matching and diffusion models, 2025.

[3] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling, 2023.

[4] Yaron Lipman, Marton Havasi, Peter Holderrieth, Neta Shaul, Matt Le, Brian Karrer, Ricky T. Q. Chen, David Lopez-Paz, Heli Ben-Hamu, and Itai Gat. Flow matching guide and code, 2024.

[5] William Peebles and Saining Xie. Scalable diffusion models with transformers, 2023.

Appendix

A Verification of the Constructed Marginal Velocity

Under our construction $x_t = (1-t)x_0 + tz$, the trajectory of each sample is deterministic and given by a simple affine transformation. Such deterministic transform preserves the total probability mass: the density is only shifted and rescaled. Consequently, the evolution of $p_t(x_t \mid z)$ must satisfy the continuity equation:

$$\frac{\partial}{\partial} p_t(x_t \mid z) + \nabla_{x_t} \cdot (p_t(x_t \mid z) \cdot v(x_t, t \mid z)) = 0.$$

Consider the marginal probability density:

$$\begin{aligned}\frac{\partial}{\partial t} p_t(x_t) &= \frac{\partial}{\partial t} \int p_t(x_t \mid z) p_{\text{target}}(z) dz \\ &= \int \frac{\partial}{\partial t} p_t(x_t \mid z) p_{\text{target}}(z) dz.\end{aligned}$$

Substitute $\dfrac{\partial}{\partial t} p_t(x_t \mid z)$ with the continuity equation:

$$\begin{aligned}\frac{\partial}{\partial t} p_t(x_t) &= \int -\nabla_{x_t} \left(p_t(x_t \mid z) \cdot v(x_t, t \mid z)\right) p_{\text{target}}(z)\, dz \\ &= -\nabla_{x_t} \cdot \int v(x_t, t \mid z) \cdot p_t(x_t \mid z) p_{\text{target}}(z)\, dz \\ &= -\nabla_{x_t} \cdot \int v(x_t, t \mid z) \frac{p_t(x_t \mid z) p_{\text{target}}(z)}{p_t(x_t)} p_t(x_t) dz \\ &= -\nabla_{x_t} \cdot \left(\left(\int v(x_t, t \mid z) \frac{p_t(x \mid z) p_{\text{target}}(z)}{p_t(x_t)} dz \right) \cdot p_t(x_t) \right) \\ &= -\nabla_{x_t} \cdot \left(v^{\text{tgt}}(x_t, t) \cdot p_t(x_t) \right).\end{aligned}$$

B Conclusion of the Linear-Form Error

We show that whenever the error term is a linear function of the target velocity $v^{\text{tgt}}(x_t, t)$, the same training gradient can be obtained by substituting it with the conditional velocity $v(x_t, t \mid z)$. Formally, assume

$$\text{error} = A v^{\text{tgt}}(x_t, t) - b_\theta,$$

where A is a matrix independent of the learnable parameters θ, and b_θ is a function of θ.

Define the marginal and conditional loss function as:

$$\mathcal{L}_{\text{marginal}} = \mathbb{E}_{t, x_t \sim p_t} \left[\left\| b_\theta - A v^{\text{tgt}}(x_t, t) \right\|_2^2 \right] \tag{20}$$

$$\mathcal{L}_{\text{conditional}} = \mathbb{E}_{t, z \sim p_{\text{target}}, x_t \sim p_t(\cdot \mid z)} \left[\left\| b_\theta - A v(x_t, t \mid z) \right\|_2^2 \right] \tag{21}$$

The expectations of $\|b_\theta\|_2^2$ under both measures coincide. Besides, $\|A v^{\text{tgt}}(x_t, t)\|_2^2$ and $\|A v(x_t, t \mid z)\|_2^2$ are inde-

pendent of θ. And we show that:

$$\begin{aligned}
\mathbb{E}_{t,x_t\sim p_t}\left[b_\theta^\top A v^{\text{tgt}}(x_t,t)\right] &= \iint b_\theta^\top A v^{\text{tgt}}(x_t,t) p_t(x_t)\,dt\,dx_t \\
&= \iint b_\theta^\top A \left(\int v(x_t,t\mid z)\frac{p_t(x_t\mid z)p_{\text{target}}(z)}{p_t(x_t)}\,dz\right) p_t(x_t)\,dt\,dx_t \\
&= \iiint b_\theta^\top A v(x_t,t\mid z) p_t(x_t\mid z) p_{\text{target}}(z)\,dz\,dt\,dx_t \\
&= \mathbb{E}_{t,z\sim p_{\text{target}},x_t\sim p_t(\cdot\mid z)}\left[b_\theta^\top A v(x_t,t\mid z)\right]
\end{aligned}$$

This shows that $\mathcal{L}_{\text{marginal}}$ and $\mathcal{L}_{\text{conditional}}$ yield identical gradients with respect to θ whenever the error depends linearly on $v^{\text{tgt}}(x_t,t)$.

C MeanFlow with Classifier-Free Guidance

We extend MeanFlow with classifier-free guidance (CFG) by modifying the marginal velocity field. Specifically, the CFG-enhanced marginal velocity is defined as:

$$v_{\text{cfg}}(x_r,r\mid c) = \omega v(x_r,r\mid c) + (1-\omega)u(x_r,r,r),$$

where c denotes the conditioning label, and $u(x_r,r,r)$ denotes the unguided instantaneous marginal velocity. Since there's no closed-form of $u(x_r,r,t)$, we use a neural network approximation $u^\theta_{\text{cfg}}(x_r,r,t)$, leading to:

$$v_{\text{cfg}}(x_r,r\mid c) = \omega v(x_r,r\mid c) + (1-\omega)u^\theta_{\text{cfg}}(x_r,r,r). \tag{22}$$

Substituting $v^{\text{tgt}}(x_r,r)$ in Equation (14) with $v_{\text{cfg}}(x_r,r\mid c)$, we derive the target CFG average velocity field to train $u^\theta_{\text{cfg}}(x_r,r,t\mid c)$:

$$u^{\text{tgt}}_{\text{cfg}}(x_r,r,t) = v_{\text{cfg}}(x_r,r\mid c) + (t-r)\left(\frac{\partial u^\theta_{\text{cfg}}}{\partial x_r}v_{\text{cfg}}(x_r,r\mid c) + \frac{\partial u^\theta_{\text{cfg}}}{\partial r}\right) \tag{23}$$

The error term $u^\theta_{\text{cfg}}(x_r,r,t) - u^{\text{tgt}}_{\text{cfg}}(x_r,r,t)$ is linear with respect to the marginal velocity $v(x_r,r\mid c)$. Following conclusion in Appendix B, we replace the marginal velocity $v_{\text{cfg}}(x_r,r\mid c)$ in Equation (23) with the conditional CFG velocity $v_{\text{cfg}}(x_r,r\mid c,z)$. Since $v(x_r,r\mid c,z) = v(x_r,r\mid z) = z-x$, the conditional CFG velocity becomes:

$$\begin{aligned}
v_{\text{cfg}}(x_r,r\mid c,z) &= \omega v(x_r,r\mid c,z) + (1-\omega)u^\theta_{\text{cfg}}(x_r,r,r) \\
&= \omega v(x_r,r\mid z) + (1-\omega)u^\theta_{\text{cfg}}(x_r,r,r),
\end{aligned} \tag{24}$$

This leads to the final results in Equation (18) and Equation (19).

D Learning Rate and Loss During Training

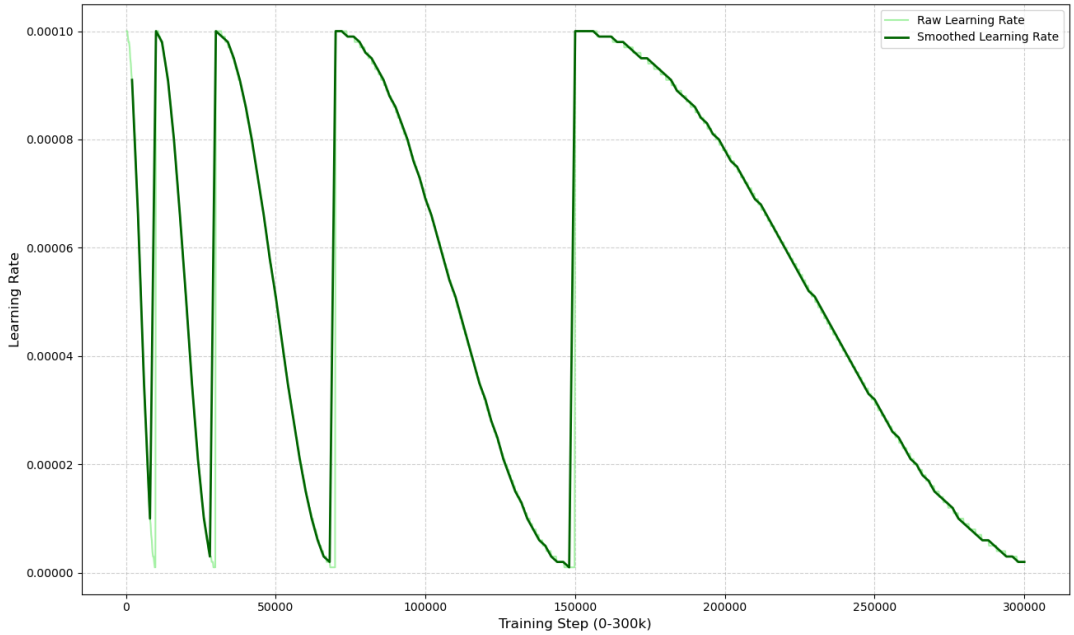

Figure 8: Learning rate over training.

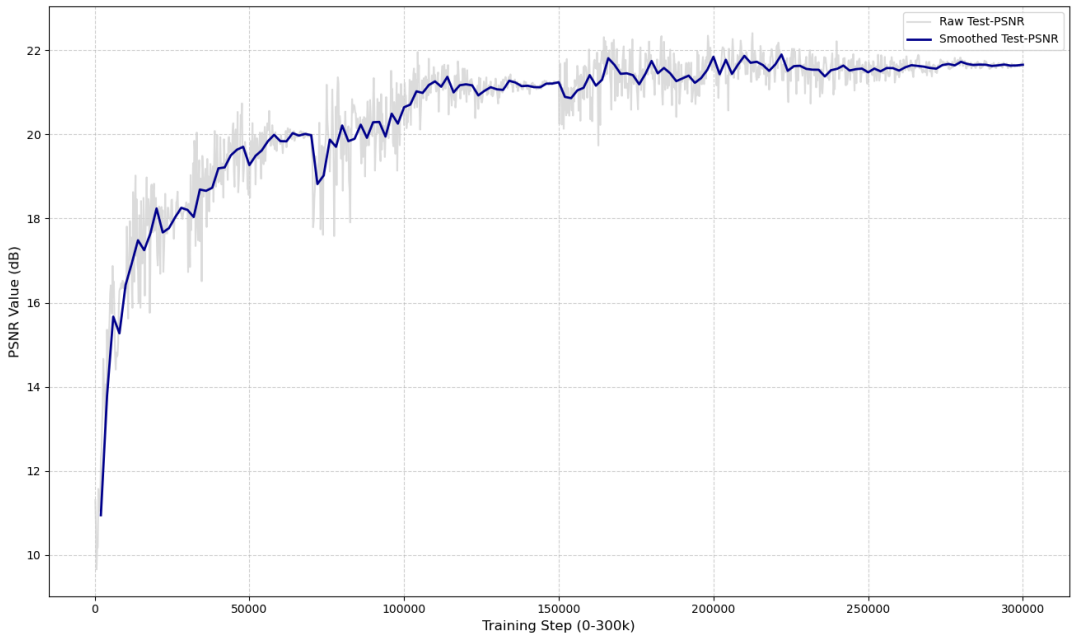

Figure 9: Loss and MSE loss over training.